

Python 函数与模块工程

把脚本重构为边界清晰、可测试、可复用的 Python 模块。

- 控制流收束到函数
- 函数声明输入输出
- 模块公开稳定接口
- 测试覆盖边界条件

控制流的可测试边界

复杂分支应拆成能够独立输入和断言输出的小函数。

- 减少隐藏状态
- 提前处理异常分支
- 保持主路径清晰
- 用参数表达变化

函数输入输出契约

函数契约同时包含有效输入、返回结果和失败语义。

```
def normalize_score(value: float) -> float:  
    if not 0 <= value <= 100:  
        raise ValueError('score out of range')  
    return value / 100
```

作用域、闭包与引用

理解局部变量、闭包捕获和可变对象引用，避免隐藏共享状态。

- 局部优先
- 谨慎捕获可变对象
- 显式传递依赖
- 避免模块级可变状态

模块公开接口

使用小而稳定的公开 API 隔离内部实现变化。

- `__all__` 声明公开符号
- 内部函数以下划线命名
- 调用方只依赖公开接口
- 文档描述契约

项目结构与依赖方向

业务逻辑不应反向依赖 CLI、HTTP 或数据库等外层适配器。

- domain 放核心规则
- adapters 放外部交互
- tests 对准公开边界
- 配置从入口注入

重构为可测试模块

将读取、转换、输出拆分，使用依赖注入替换真实外部系统。

```
def run(load, save):  
    records = load()  
    result = transform(records)  
    save(result)
```

异常契约作为模块边界

异常类型是模块 API 的一部分，调用方应只处理公开的失败类别。

- 捕获最窄异常
- 保留原始原因
- 转换为领域错误
- 不要静默吞掉失败